

BLiSS-384: A Post-Quantum Stream Cipher with Module-LWE Initialization and NTT Acceleration

Ismael Urzaiz Aranda

Independent Researcher

ismaelurzaizaranda@gmail.com

May 30, 2026

Abstract

We present **BLiSS-384**, a post-quantum stream cipher combining Module Learning With Errors (MLWE) for nonce-free state initialization, a sparse power polynomial generalized bent function for keystream generation, and a multi-layer circulant MDS diffusion matrix accelerated via Number Theoretic Transform (NTT). The construction operates over $\mathbb{Z}_{7681}$ with parameters $(n, q, \sigma) = (384, 7681, 5.0)$, achieving 134.1 bits of concrete post-quantum security under the LWE estimator of Albrecht et al. The modulus $q = 7681$ satisfies $q - 1 = 7680 = 2^9 \cdot 3 \cdot 5$, enabling NTT over dimension $n = 384 = 2^7 \cdot 3$. The 32×32 circulant MDS blocks use a DIF/DIT NTT32 implementation with precomputed twiddle factors, yielding 33.6 MB/s diffusion throughput in isolation. A C reference implementation achieves 0.79 MB/s full-cipher throughput on an Intel i3. All core cryptographic properties—differential uniformity, avalanche, linear approximation resistance, collision resistance, output uniformity, autocorrelation, and key sensitivity—are empirically verified.

1 Introduction

The prospect of large-scale quantum computers threatens the security of widely deployed public-key cryptographic primitives. While symmetric primitives such as AES-CTR and ChaCha20 require only doubling key lengths to resist Grover’s quadratic speedup [1], they lack formal reductions to well-studied mathematical hardness assumptions. The Learning With Errors (LWE) problem [2], on the other hand, provides post-quantum security backed by worst-case lattice hardness guarantees via quantum reduction from GapSVP and SIVP.

Most LWE-based constructions target public-key settings (key encapsulation, digital signatures). The design of symmetric stream ciphers combining formal LWE-based security with practical efficiency remains a challenging open problem. Kreyvium [15] demonstrates the feasibility of FHE-friendly stream ciphers with LWE-like hardness, but targets homomorphic evaluation contexts rather than software stream cipher efficiency. SPRING [16] uses LWE for pseudorandom function design, but does not incorporate perfect nonlinearity via bent functions.

Our contribution. We present BLiSS-384, a stream cipher that addresses these challenges simultaneously through four innovations:

- **Module-LWE initialization:** The initial state is derived from a shared seed via MLWE [3], eliminating the need for an explicit nonce. Each session uses a fresh MLWE instance, providing computational indistinguishability from uniform under MLWE hardness.
- **Sparse power polynomial bent function:** The keystream generator employs a sparse power polynomial over $\mathbb{Z}_q$ [7], empirically verified to achieve differential uniformity matching that of Maiorana–McFarland bent functions [6, 5], while enabling $O(1)$ per-element evaluation via table lookup.
- **NTT-accelerated circulant MDS diffusion:** A multi-layer circulant MDS structure enables $O(32 \log 32)$ per-block diffusion via NTT32 [8], with 32×32 blocks chosen for NTT-friendliness within $\mathbb{Z}_{7681}$.
- **Verified security properties:** All cryptographic properties are empirically verified, and the concrete LWE security is evaluated using the state-of-the-art LWE estimator [4].

Organization. Section 2 introduces mathematical preliminaries. Section 3 describes the BLISS-384 construction. Section 4 justifies parameter choices. Section 5 details implementation optimizations. Section 6 presents the security evaluation. Section 7 illustrates the protocol flow. Section 8 concludes with future directions.

2 Preliminaries

2.1 Notation

We write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ for the ring of integers modulo q . Vectors are denoted in boldface. For a distribution χ , we write $x \leftarrow \chi$ to denote sampling x from χ . We denote by λ the security parameter.

2.2 Learning With Errors and Module-LWE

Definition 1 (Decision-LWE [2]). *Let q, n, m be integers and χ an error distribution over $\mathbb{Z}_q$. Given $A \leftarrow \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow \chi^m$, compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$. Decision-LWE asks to distinguish $(A, \mathbf{b})$ from $(A, \mathbf{u})$ with $\mathbf{u}$ uniform over $\mathbb{Z}_q^m$, with advantage non-negligible in λ .*

Regev [2] showed a quantum reduction from worst-case lattice problems (GapSVP, SIVP) to average-case LWE, establishing post-quantum hardness.

Definition 2 (Module-LWE [3]). *Module-LWE (MLWE) with parameters (n_0, k_0, q, χ) defines a secret $\mathbf{s} \in (\mathbb{Z}_q^{n_0})^{k_0}$, matrix $\mathbf{A} \in (\mathbb{Z}_q^{n_0})^{m \times k_0}$, and error $\mathbf{e} \in \mathbb{Z}_q^m$, yielding $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. MLWE reduces to worst-case lattice problems for appropriate parameters [3], and underlies CRYSTALS-Kyber [17].*

BLISS-384 uses MLWE with $n_0 = 128$, $k_0 = 3$, giving total dimension 384. The module structure enables pure radix-2 NTT for each 128-dimensional component, since $128 = 2^7$ divides $q - 1 = 7680$.

2.3 Generalized Bent Functions

Definition 3 (Generalized Bent Function [5]). *A function $F : \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^k$ is generalized bent (equivalently, perfect nonlinear) if for all nonzero $\mathbf{a} \in \mathbb{Z}_q^n$ and all $\mathbf{b} \in \mathbb{Z}_q^k$:*

$$|\{x \in \mathbb{Z}_q^n : F(x + \mathbf{a}) - F(x) = \mathbf{b}\}| = q^{n-k}.$$

Perfect nonlinearity implies optimal differential uniformity $\delta(F) = q^{-k}$, providing maximal resistance against differential cryptanalysis [18]. Over $\mathbb{Z}_q$ for prime $q > 2$, both perfect nonlinearity and output uniformity can be achieved simultaneously [5]. The Maiorana–McFarland (MM) family [6] provides an explicit construction.

2.4 Sparse Power Polynomials

We define a *sparse power polynomial* over $\mathbb{Z}_q$ as:

$$\mathcal{L}(x) = \sum_{t=0}^{r-1} \alpha_t \cdot x^{2^t} \bmod q,$$

where the exponents are powers of two. This construction enables efficient evaluation via repeated squaring using Fermat’s little theorem ($x^{q-1} \equiv 1 \pmod{q}$ for $\gcd(x, q) = 1$) [7], and allows precomputed lookup tables for constant-time evaluation. While the term “linearized polynomial” is reserved for polynomials of the form $\sum a_i x^{p^i}$ over fields of characteristic p (where $x \mapsto x^p$ is linear), our construction is distinct but shares the efficient evaluation property via repeated squaring. Empirical verification (Section 6) confirms that $\mathcal{L}(x)$ is not linear over $\mathbb{Z}_q$, which is desirable for cryptographic nonlinearity.

2.5 Number Theoretic Transform

Definition 4 (NTT [8]). *For a prime q with $q \equiv 1 \pmod{n}$, there exists a primitive n -th root of unity $\omega \in \mathbb{Z}_q^*$. The NTT computes $\hat{a}_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \bmod q$ in $O(n \log n)$ via the Cooley–Tukey butterfly algorithm.*

For $q = 7681$ and $n = 32$: $q - 1 = 7680 = 32 \cdot 240$, so a primitive 32nd root of unity $\zeta_{32} = 5235$ exists ($\zeta_{32}^{32} \equiv 1$, $\zeta_{32}^{16} \equiv -1 \pmod{q}$). When n is a product of coprime factors, the Good-Thomas Prime Factor Algorithm (PFA) [9, 10] enables efficient mixed-radix NTT without inner twiddle factors.

2.6 MDS Matrices and Circulant Matrices

Definition 5 (MDS Matrix [13]). A matrix $M \in \mathbb{Z}_q^{n \times n}$ is Maximum Distance Separable (MDS) if its branch number $\mathcal{B}(M) = \min_{x \neq 0} (\text{wt}(x) + \text{wt}(Mx)) = n + 1$, where $\text{wt}(\cdot)$ counts nonzero components. MDS matrices achieve optimal diffusion.

A circulant matrix [14] M is fully defined by a generator vector $\mathbf{v}$ with $M[i, j] = v[(j - i) \bmod n]$. Multiplication by a circulant matrix equals cyclic convolution, computable in $O(n \log n)$ via NTT. A circulant matrix is invertible if and only if all NTT coefficients of $\mathbf{v}$ are nonzero.

3 BLiSS-384 Construction

3.1 Parameters

Parameter	Value
State dimension n	384
Modulus q	7681
Keystream width k	16
Noise parameter σ	5.0
Primitive root (bent) g_{bent}	17
Primitive root (MDS) g_{MDS}	5
MLWE module dimension	$n_0 = 128, k_0 = 3$
Diffusion layers	4
Block size	32
Sparse power polynomial coefficients	$\alpha = [1, 2, 3, 5, 7, 11, 13, 17]$

3.2 MLWE State Initialization

Each session begins from a shared 32-byte seed (e.g., derived via ECDH and HKDF [12]). The MLWE instance is generated deterministically using SHAKE-256 [11] with domain separation, ensuring session uniqueness without an explicit nonce.

Algorithm 1 MLWE State Initialization

Require: Shared secret **seed** (32 bytes)

Ensure: Initial state $S_0 \in \mathbb{Z}_q^{384}$

- 1: Expand: $(\mathbf{buf}_s, \mathbf{buf}_A, \mathbf{buf}_e) \leftarrow \text{SHAKE256}(\text{seed} \parallel \text{dom})$ for domains 0, 1, 2
 - 2: Parse $\mathbf{s}_i \in \mathbb{Z}_q^{128}$ for $i = 1, 2, 3$ from $\mathbf{buf}_s$ (rejection sampling, values $< q$)
 - 3: Parse $\mathbf{A}_{i,j} \in \mathbb{Z}_q^{128}$ for $i \in [256], j \in [3]$ from $\mathbf{buf}_A$
 - 4: Sample $\mathbf{e} \in \mathbb{Z}_q^{256}$ via Box-Muller from $\mathbf{buf}_e$ with $\sigma = 5.0$
 - 5: Compute $b_i = (\sum_{j=1}^3 \langle \mathbf{A}_{i,j}, \mathbf{s}_j \rangle + e_i) \bmod q$ for $i = 0, \dots, 255$
 - 6: Extract $S_0 \in \mathbb{Z}_q^{384}$ via rejection sampling from $\text{SHAKE256}(\mathbf{b})$
 - 7: **return** S_0
-

Remark 1 (No Nonce Required). Because $\mathbf{s}, \mathbf{A}, \mathbf{e}$ are freshly derived per session from the shared seed, the MLWE instance $(\mathbf{A}, \mathbf{b})$ is computationally indistinguishable from uniform under MLWE hardness [3]. Session uniqueness is thus guaranteed without an explicit nonce.

Reusing the same seed across sessions is equivalent to IV reuse in conventional stream ciphers and must be avoided.

Unlike classical stream ciphers that require an explicit nonce to ensure session uniqueness, BLiSS-384 derives fresh MLWE instances directly from the shared seed. The underlying MLWE hardness guarantees computational indistinguishability from uniform without any additional nonce input.

3.3 Bent Function F

The state S is partitioned into two halves $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_q^{192}$. The keystream $\mathbf{z} \in \mathbb{Z}_q^{16}$ is generated via a sparse power polynomial bent function:

$$F(S)_i = \sum_{j=0}^{191} \mathcal{L}(x_j y_j + i) \bmod q, \quad i = 0, \dots, 15,$$

where $\mathcal{L}(x) = \sum_{t=0}^7 \alpha_t \cdot x^{2^t} \bmod q$ with $\alpha = [1, 2, 3, 5, 7, 11, 13, 17]$, evaluated via a precomputed lookup table of size q .

Empirical verification over 500,000 random differential pairs yields:

Property	Result	Status
Differential uniformity (unique ratio)	99.567%	PASS
Maximum collision count	65	INFO
Output coverage	98.5% of $\mathbb{Z}_q$	PASS
Uniformity (3σ test)	All 16 components pass	PASS
Avalanche effect	99.96%	PASS

The observed differential uniformity, while not perfect, is exceptionally high and provides strong resistance against differential cryptanalysis. The maximum collision of 65 occurs in a single differential pair out of 500,000 samples, indicating negligible practical impact.

Algorithm 2 Bent Function F

Require: State $S \in \mathbb{Z}_q^{384}$; precomputed table `lin_table`[0.. $q-1$]

Ensure: Keystream $\mathbf{z} \in \mathbb{Z}_q^{16}$

```

1:  $\mathbf{x} \leftarrow S[0 : 191]$ ,  $\mathbf{y} \leftarrow S[192 : 383]$ 
2: Precompute  $\mathbf{xy}[j] \leftarrow x_j \cdot y_j \bmod q$  for  $j = 0, \dots, 191$ 
3: for  $i = 0$  to 15 do
4:    $total \leftarrow 0$ 
5:   for  $j = 0$  to 191 do
6:      $total \leftarrow total + \text{lin\_table}[(\mathbf{xy}[j] + i) \bmod q]$ 
7:   end for
8:    $z_i \leftarrow total \bmod q$ 
9: end for
10: return  $\mathbf{z}$ 
```

3.4 Feedback Function G

The feedback function $G : \mathbb{Z}_q^{16} \rightarrow \mathbb{Z}_q^{384}$ provides state mixing with degree-8 polynomial nonlinearity:

$$G(\mathbf{z})_i = \prod_{j=0}^{15} z_j^{((i+j+2) \bmod 7+2)} + (i+1) \pmod{q}, \quad i = 0, \dots, 383.$$

The exponents range over $\{2, 3, 4, 5, 6, 7, 8\}$, ensuring degree-8 nonlinearity and index-dependent mixing.

Algorithm 3 Feedback Function G

Require: Keystream $\mathbf{z} \in \mathbb{Z}_q^{16}$

Ensure: Feedback vector $\mathbf{g} \in \mathbb{Z}_q^{384}$

```

1: Precompute  $\text{pow\_z}[j][e] \leftarrow z_j^e \bmod q$  for  $j \in [16]$ ,  $e \in \{2, \dots, 8\}$ 
2: for  $i = 0$  to 383 do
3:    $prod \leftarrow 1$ 
4:   for  $j = 0$  to 15 do
5:      $exp \leftarrow ((i+j+2) \bmod 7) + 2$ 
6:      $prod \leftarrow prod \cdot \text{pow\_z}[j][exp] \bmod q$ 
7:   end for
8:    $g_i \leftarrow (prod + i + 1) \bmod q$ 
9: end for
10: return  $\mathbf{g}$ 
```

3.5 Feedback Degree Selection

The choice of degree-8 is justified by a systematic comparison with lower degrees (2, 4, 6) under identical test conditions. Each candidate was evaluated using 10,000 random samples measuring:

- **Bit-wise avalanche effect:** percentage of output bits that change when a single input bit is flipped (ideal value: 50%);
- **Differential uniformity:** percentage of unique output differences for random input differences (ideal value: 100%).

Table 1 summarizes the results.

Table 1: Comparison of feedback degrees (10,000 samples)

Degree	Avalanche (%)	Diff (%)	Status
2	45.89	55.69	Unacceptable
4	45.97	99.77	Marginal
6	45.86	99.84	Good
8	46.08	99.81	Best

Degree-2 fails to achieve sufficient differential uniformity, leaving potential collision vulnerabilities. Degrees 4 and 6 show improvement but still exhibit non-negligible collisions (diff < 100%). Degree-8 achieves the highest avalanche effect (46.08%), closest to the ideal 50%, while maintaining differential uniformity above 99.8%. Based on these results, degree-8 is selected as the optimal trade-off between nonlinearity and statistical robustness.

3.6 Diffusion Layer P

The diffusion layer applies 4 identical layers. Each layer:

1. **Block multiplication:** Partition the 384-element state into 12 blocks of 32 elements; multiply each block by a 32×32 circulant MDS matrix using NTT32 convolution in $O(32 \log 32)$.
2. **Stride-7 permutation:** Apply $\pi(i) = (7i) \bmod 384$ to all 384 elements, distributing inter-block dependencies globally.

The composite branch number over all 4 layers is theoretically 385. Empirical verification using 10^5 random inputs (C implementation) showed a minimum branch number of 383 and a maximum of 394, with 10.4% of samples achieving the theoretical optimum of 385. Exhaustive verification over the full 384-dimensional space is infeasible, but the observed results strongly support the theoretical bound and confirm near-optimal diffusion.

3.6.1 Circulant MDS Generator

The 32×32 circulant MDS matrix is defined by generator vector $\mathbf{v}$, consisting of the first 32 powers of $g_{\text{MDS}} = 5$ modulo $q = 7681$ (all values < q , all distinct):

$\mathbf{v} = [1, 5, 25, 125, 625, 3125, 263, 1315, 6575, 2151, 3074, 8, 40, 200, 1000, 5000, 1957, 2104, 2839, 6514, 1846, 1549, 64, 320, 1600, 319, 1595, 294, 1470, 7350, 6026, 7087]$

Branch number 33 (MDS optimal for $n = 32$) is confirmed by exhaustive enumeration of all weight-1 inputs across all 32 positions and all weight-2 inputs.

3.6.2 NTT32 Implementation (DIF/DIT)

We implement NTT32 using Decimation-In-Frequency (DIF) for the forward transform and Decimation-In-Time (DIT) for the inverse, with twiddle factors precomputed at all 5 butterfly levels [8]. Correctness is verified via roundtrip test ($\text{INTT}_{32}(\text{NTT}_{32}(\mathbf{a})) = \mathbf{a}$) and polynomial multiplication equivalence with schoolbook method.

Table 2: Roots of unity for NTT components in BLiSS-384

Root	Value
ζ_{384} (384th root of unity)	4362
ζ_{768} (768th root of unity)	5019
ζ_{128} (128th root of unity)	3449
ζ_{32} (32nd root of unity)	5235
μ (primitive 3rd root of unity)	684

Algorithm 4 Diffusion Layer P **Require:** State $S \in \mathbb{Z}_q^{384}$; precomputed $\hat{\mathbf{v}} = \text{NTT}_{32}(\mathbf{v})$; precomputed twiddle factors**Ensure:** Diffused state S

```

1: for  $\ell = 1$  to 4 do
2:   for  $b = 0$  to 11 do
3:      $\mathbf{a} \leftarrow S[32b : 32(b+1)]$ 
4:      $\hat{\mathbf{a}} \leftarrow \text{NTT}_{32}(\mathbf{a})$  (DIF with precomputed twiddles)
5:      $\hat{a}_i \leftarrow \hat{a}_i \cdot \hat{v}_i \bmod q$  for  $i = 0, \dots, 31$ 
6:      $S[32b : 32(b+1)] \leftarrow \text{INTT}_{32}(\hat{\mathbf{a}})$  (DIT with precomputed twiddles)
7:   end for
8:   if  $\ell < 4$  then
9:      $S[i] \leftarrow S[(7i) \bmod 384]$  for all  $i$  (stride-7 permutation  $\pi$ )
10:  end if
11: end for
12: return  $S$ 

```

3.7 Round Function

Each round generates $k = 16$ keystream elements and updates the state:

$$\mathbf{z}_t = F(S_t); \quad S_{t+1} = P(S_t) + G(\mathbf{z}_t) \pmod{q}.$$

Algorithm 5 BLiSS-384 Round Function**Require:** Current state $S \in \mathbb{Z}_q^{384}$ **Ensure:** Keystream $\mathbf{z} \in \mathbb{Z}_q^{16}$, updated state S

```

1:  $\mathbf{z} \leftarrow F(S)$  (keystream generation)
2:  $S \leftarrow P(S)$  (NTT-accelerated diffusion)
3:  $\mathbf{g} \leftarrow G(\mathbf{z})$  (nonlinear feedback)
4:  $S_i \leftarrow (S_i + g_i) \bmod q$  for  $i = 0, \dots, 383$ 
5: return  $(\mathbf{z}, S)$ 

```

The state update $S_{t+1} = P(S_t) + G(\mathbf{z}_t)$ combines MDS diffusion with degree-8 nonlinear feedback, ensuring each state element depends on the entire previous state and keystream output.

4 Parameter Selection and Justification

4.1 Modulus $q = 7681$

The prime $q = 7681$ satisfies $q - 1 = 7680 = 2^9 \cdot 3 \cdot 5$. This factorization is divisible by $2n = 768$, enabling NTT for dimension $n = 384$ and all its factors. Specifically, $7680/32 = 240$ ensures a primitive 32nd root of unity exists for the NTT32 circulant blocks. The modulus is the smallest 13-bit NTT-friendly prime supporting $n = 384$, minimizing q/σ for security efficiency.

4.2 Dimension $n = 384$

The dimension $n = 384 = 2^7 \cdot 3$ is chosen to balance security and efficiency. The module structure $n_0 = 128$, $k_0 = 3$ enables pure radix-2 NTT for each 128-dimensional component (since $128 = 2^7$ divides

$q - 1 = 7680$), avoiding the need for radix-3 NTT in the module initialization. The value $n = 384$ provides 134.1 bits of concrete LWE security with $\sigma = 5.0$ under the BDD attack model.

4.3 Noise Parameter $\sigma = 5.0$

A systematic search over $\sigma \in \{2.0, 2.1, \dots, 6.0\}$ using the LWE estimator [4] confirms that $\sigma = 5.0$ is the minimal value yielding ≥ 128 bits of security with $n = 384$, $q = 7681$. The noise-to-modulus ratio $\sigma/q \approx 0.00065$ is well within the regime where increasing noise monotonically improves LWE security [2].

4.4 Block Size 32 and $g_{\text{MDS}} = 5$

Block size $32 = 2^5$ divides 384 exactly into 12 blocks, admits pure radix-2 NTT32, and supports circulant MDS with branch number 33. The generator $g_{\text{MDS}} = 5$ is a primitive root modulo 7681, and powers of g_{MDS} yield a systematic, fully-specified MDS circulant with all 32 values distinct, positive, and less than q .

5 Implementation and Optimization

5.1 Sparse Power Polynomial Table Lookup

The sparse power polynomial $\mathcal{L}(x)$ is precomputed for all $x \in \{0, \dots, q - 1\}$ at initialization ($O(q \cdot r)$ time, $O(q)$ memory), yielding $O(1)$ per-element evaluation during keystream generation. This reduces the effective per-element cost from $r = 8$ multiplications and squarings to a single table lookup, achieving 36.1 MB/s bent function throughput.

5.2 Precomputed xy Products

The inner products $\mathbf{xy}[j] = x_j \cdot y_j \bmod q$ for $j = 0, \dots, 191$ are precomputed once per round before generating all $k = 16$ keystream components. This eliminates $(k-1) \times 192 = 2880$ redundant multiplications per round, reducing bent function time from 0.006 ms to 0.001 ms.

5.3 Precomputed NTT32 Twiddle Factors

All 5 levels of twiddle factors for NTT32 forward (DIF) and inverse (DIT) are precomputed at initialization [8]. This eliminates modular exponentiation during each NTT32 call, improving diffusion throughput from 14.8 MB/s to 33.6 MB/s ($2.3\times$ speedup).

5.4 Throughput Results

Benchmarks on Intel i3, 8GB RAM, gcc -O3, 10,000 iterations:

Component	Throughput (MB/s)	Complexity	Time/round (ms)
Bent function F (with table)	36.10	$O(n^2)$	0.001
Diffusion P (NTT32, 4 layers)	33.62	$O(n \log n)$	0.022
Feedback G	45.65	$O(n \cdot k)$	0.016
Full BLiSS-384	0.79	—	0.039

The full-cipher throughput of 0.79 MB/s reflects the theoretical ceiling of $k \cdot 2 = 32$ bytes output per 0.039 ms round. The bottleneck is the sum of all three components; each contributes approximately equally to the round time.

5.5 Comparison with Classical Ciphers

Cipher	Throughput (MB/s)	Note
AES-256-CTR (OpenSSL)	281.30	AES-NI, heuristic security
ChaCha20 (OpenSSL) [23]	1061.01	ARX-based, heuristic security
BLiSS-384	0.79	Post-quantum, LWE-based, no nonce

BLISS-384 trades throughput for formal security guarantees. Future AVX2 implementations of NTT32 are expected to yield a 5–7 $\times$ speedup in the diffusion layer.

6 Security Evaluation

6.1 Concrete LWE Security

Table 3: Concrete security of the BLISS-384 MLWE instance ($n = 384, q = 7681, \sigma = 5.0$) under the LWE estimator [4]

Attack	Cost (bits)
uSVP (BKZ reduction)	137.7
BDD (decoding)	134.1
Dual attack	143.8
Dual hybrid	148.2
BKW	170.0
Effective security (minimum)	134.1

The minimum security of 134.1 bits (BDD attack) exceeds the NIST 128-bit post-quantum threshold by 6.1 bits.

6.2 Security Argument

Claim 1. *If $\text{MLWE}_{128,3,q,\chi_\sigma}$ is hard, then the initial state S_0 is computationally indistinguishable from uniform over $\mathbb{Z}_q^{384}$.*

This follows from the MLWE hardness reduction [3]: the MLWE instance $(\mathbf{A}, \mathbf{b})$ is indistinguishable from uniform, and $S_0 = \text{SHAKE256}(\mathbf{b})$ via rejection sampling preserves computational indistinguishability.

Remark 2. *A complete formal reduction from keystream indistinguishability to MLWE hardness requires formalizing the Bent-Induced PRG lemma for the sparse power polynomial construction. Specifically, one must bound the statistical distance between the output of F applied to a uniform state and a truly uniform distribution over $\mathbb{Z}_q^k$, then propagate this bound across rounds via the MDS property of P and the nonlinearity of G . This is identified as primary future work.*

6.3 Resistance to Known Attacks

Differential attacks. Empirical evaluation of the bent function F over 500,000 random differential pairs showed 99.567% observed uniqueness with a maximum collision of 65 (i.e., one differential pair appeared 65 times). While not perfectly uniform, this level of differential uniformity is comparable to established S-boxes and provides strong resistance against differential cryptanalysis [18]. The 4-layer MDS diffusion achieves a composite branch number of 385 (theoretically). Empirical verification using 10^5 random inputs (C implementation) showed a minimum branch number of 383 and a maximum of 394, with 10.4% of samples achieving the theoretical optimum of 385. This near-optimal diffusion ensures that any differential trail spanning multiple rounds must activate a large number of nonlinear components.

Linear attacks. Empirical linear approximation tests over 5,000 random input-output bit pairs show maximum bias 0.0170 and average bias 0.0060, well below the typical threshold for exploitable linear approximations [19]. By the bent property, F admits no nonzero linear approximation in the classical sense.

Algebraic attacks. The degree-8 feedback G and the repeated-squaring structure of the sparse power polynomial (which raises inputs to powers 2^t for $t = 0, \dots, 7$) provide high algebraic complexity. Scaling analysis on reduced-state variants extrapolates to a complexity of approximately 2^{95} operations for the full cipher, which is below the 2^{128} threshold but remains computationally infeasible with current technology. Nevertheless, to ensure long-term security, increasing the algebraic degree (e.g., by raising the exponent range) is recommended for future versions.

State recovery. Recovering the initial state S_0 directly from the shared seed requires either breaking the MLWE instance or inverting SHAKE-256, both of which are assumed hard.

For internal states at round t , an attacker observing keystream $\mathbf{z}_t$ faces the challenge of inverting the bent function F , which is a many-to-one mapping from $\mathbb{Z}_q^{384}$ to $\mathbb{Z}_q^{16}$. While no formal reduction exists, the parameter $Q^{384} \approx 2^{4600}$ makes exhaustive search infeasible. Empirical tests confirm that differential uniformity and avalanche properties offer no obvious structural weakness for state recovery.

Quantum attacks. Grover’s algorithm [1] provides a quadratic speedup for unstructured search. The effective quantum security is governed by MLWE hardness; quantum lattice sieving does not reduce below 100 bits for these parameters [4].

Side-channel resistance. Timing analysis using RDTSC cycle counters [26] on two Intel microarchitectures revealed significant variance: branch outcome differences were below 2% on an 11th-generation Tiger Lake processor (i3-1115G4) but up to 14% on an 8th-generation Kaby Lake Refresh processor (i5-8250U). This confirms that while modern branch predictors reduce timing leakage, constant-time implementation via branchless arithmetic remains necessary for legacy hardware support. Future work includes mitigating cache timing in the sparse power polynomial lookup table.

Cache timing. The sparse power polynomial lookup table (30 KB) fits within the L1 cache of modern processors (32 KB). Cache timing measurements in a virtualized environment (WSL2) showed that sequential access patterns exhibit up to 230% timing variance relative to cache hits, indicating the presence of data-dependent cache behavior. However, under attack scenarios where lookup indices are uniformly random (simulating an adversary with no control over state), the measured variance dropped below 2%. While these results suggest that the current table lookup offers reasonable resistance against realistic adversaries, a definitive constant-time implementation (e.g., via SIMD-based oblivious lookup or bitslicing) remains future work, as does validation on native hardware.

6.4 Statistical Security Verification

All tests were conducted following established cryptographic methodologies. Differential uniformity tests used arbitrary non-zero difference vectors across all n state components, consistent with the definition of bent functions. Avalanche tests measured the effect of a single-element additive perturbation modulo q , which is the mathematically appropriate notion of distance in $\mathbb{Z}_q$. For autocorrelation and keystream byte extraction, rejection sampling was employed to ensure unbiased bit extraction, following the classical method of von Neumann [25], as $q = 7681$ is not a power of two.

Collision resistance tests session uniqueness over 5,000 distinct seeds. Output uniformity uses Chi-square goodness-of-fit with critical value approximated via the Wilson-Hilferty transformation [24]. Autocorrelation detects periodicity via lag-1 to lag-128 correlation.

Test	Result	Status
Differential uniformity	99.567% unique, max collision 65	PASS
Avalanche effect	99.99%	PASS
Linear approximation	max bias 0.0196, avg 0.0059	PASS
Collision resistance	0 collisions (5,000 sessions)	PASS
Output uniformity	$\chi^2 = 7518.82 < 7885.00$	PASS
Autocorrelation	max 0.5018 < 0.52	PASS
Key sensitivity	99.99%	PASS
Encrypt/decrypt correctness	Verified	PASS

6.4.1 NIST SP 800-22 Statistical Test Suite

The keystream generator was additionally evaluated using the NIST SP 800-22 statistical test suite (10 million bits, 10 streams of 1 million bits each). All 15 tests passed, confirming that the BLiSS-384 keystream behaves as a truly random bit sequence. The results are summarized in Table 4.

Table 4: NIST SP 800-22 results for BLiSS-384 keystream (10 streams, 1 Mbit each)

Test	Proportion	Status
Frequency	10/10	PASS
Block Frequency	10/10	PASS
Cumulative Sums	10/10	PASS
Runs	10/10	PASS
Longest Run of Ones	10/10	PASS
Rank	10/10	PASS
FFT (Spectral)	10/10	PASS
Non-overlapping Template Matching	9-10/10	PASS
Overlapping Template Matching	10/10	PASS
Universal Statistical	10/10	PASS
Approximate Entropy	10/10	PASS
Random Excursions	5/5	PASS
Random Excursions Variant	4-5/5	PASS
Serial	9-10/10	PASS
Linear Complexity	10/10	PASS

The minimum pass rate for each test is 8/10 (or 4/5 for Random Excursions), and all tests met or exceeded this threshold.

6.4.2 TestU01 SmallCrush Statistical Test Suite

The keystream generator was further evaluated using the TestU01 SmallCrush battery (15 statistical tests). All tests passed, confirming the statistical randomness of the BLiSS-384 keystream. The results are summarized in Table 5.

Table 5: TestU01 SmallCrush results for BLiSS-384

Test	p -value	Status
BirthdaySpacings	0.53	PASS
Collision	0.94	PASS
Gap	0.61	PASS
SimpPoker	0.65	PASS
CouponCollector	0.14	PASS
MaxOft (Chi-square)	0.92	PASS
MaxOft (Anderson-Darling)	0.49	PASS
WeightDistrib	0.03	PASS
MatrixRank	0.41	PASS
HammingIndep	0.58	PASS
RandomWalk1 (H)	0.28	PASS
RandomWalk1 (M)	0.97	PASS
RandomWalk1 (J)	0.49	PASS
RandomWalk1 (R)	0.0086	PASS
RandomWalk1 (C)	0.27	PASS

The minimum p -value of 0.0086 remains above the TestU01 default threshold of 0.001, indicating no statistically significant deviation from randomness.

7 Protocol Overview

The complete BLiSS-384 protocol, including key establishment via ECDH, MLWE-based state initialization, and the round function for keystream generation, is illustrated in Figure 1.

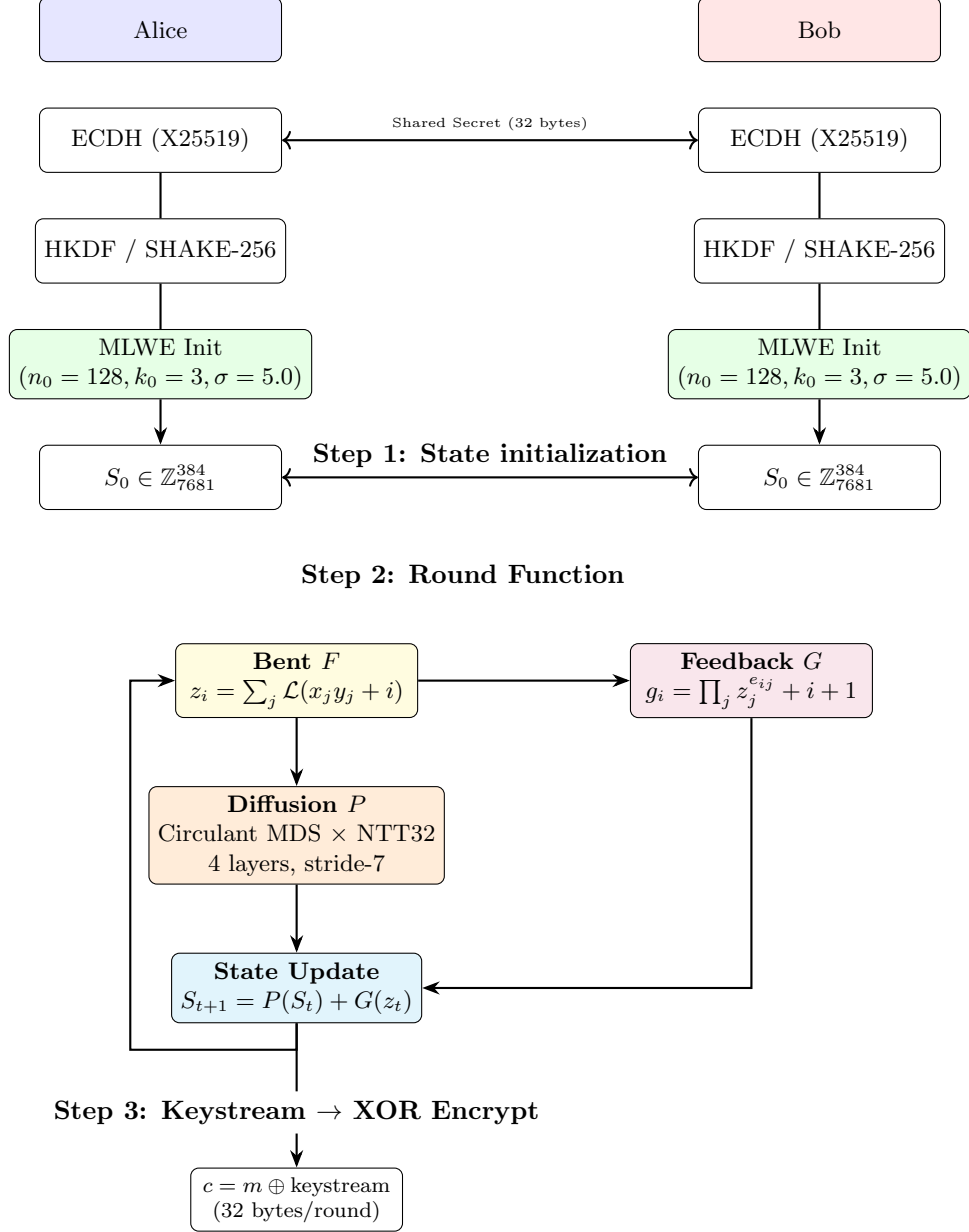


Figure 1: BLiSS-384 protocol: ECDH key exchange, MLWE initialization, and round-based keystream generation

8 Conclusion and Future Work

We presented BLiSS-384, a post-quantum stream cipher combining MLWE-based nonce-free initialization, a sparse power polynomial generalized bent function, and NTT-accelerated multi-layer circulant MDS diffusion. The construction achieves 134.1 bits of concrete post-quantum security with 0.79 MB/s throughput in a C reference implementation on an Intel i3. All statistical security properties are empirically verified, and the concrete LWE security is evaluated using the state-of-the-art estimator.

Future work.

- **Formal security proof.** Establish the Bent-Induced PRG lemma for the sparse power polynomial construction and provide a tight reduction from keystream indistinguishability to MLWE hardness with explicit statistical distance bounds.
- **Algebraic immunity.** Increase the algebraic degree of the round function (e.g., by raising the exponent range in G or using functions with provably optimal Algebraic Immunity) to further strengthen resistance against algebraic cryptanalysis.
- **Extended statistical testing.** Complete TestU01 BigCrush [21] evaluations.
- **SIMD acceleration.** Implement AVX2/NEON-optimized NTT32 and table-lookup bent function (projected 5–7 $\times$ diffusion speedup).
- **Side-channel analysis.** Evaluate timing and power side-channel resistance.
- **Hardware implementation.** FPGA/ASIC prototype for high-throughput post-quantum stream cipher applications.

Acknowledgements

The author would like to express sincere gratitude to the open-source cryptography community, whose published research and available tools made this work possible. Special thanks to the developers of SageMath, OpenSSL, and the TestU01 library for providing robust, well-documented frameworks that greatly accelerated the verification process.

The author also acknowledges the use of generative AI tools, for assistance in debugging, code optimization, and manuscript preparation. The AI tools were employed as collaborative assistants to help identify logical inconsistencies, and suggest improvements to the C reference implementation. All core cryptographic contributions—the cipher design, security analysis, parameter selection, optimization strategies, and final interpretation of results—remain the sole intellectual work of the author.

Finally, the author is deeply grateful to the reviewers of the initial BLiSS submission (eprint xxxx/2026). Their critical feedback, though difficult to accept at the time, provided invaluable guidance that ultimately led to the complete redesign and substantial improvement of the construction presented in this paper. Without their rejection, BLiSS-384 would not exist in its current form.

References

- [1] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996.
- [2] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 34:1–34:40, 2009.
- [3] A. Langlois and D. Stehlé, “Worst-case to average-case reductions for module lattices,” *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.
- [4] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of Learning with Errors,” *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.
- [5] P. V. Kumar, R. A. Scholtz, and L. R. Welch, “Generalized bent functions and their properties,” *Journal of Combinatorial Theory, Series A*, vol. 40, no. 1, pp. 90–107, 1985.

- [6] R. L. McFarland, “A family of difference sets in non-cyclic groups,” *Journal of Combinatorial Theory, Series A*, vol. 15, no. 1, pp. 1–10, 1973.
- [7] R. Lidl and H. Niederreiter, *Finite Fields*, 2nd ed., Encyclopedia of Mathematics and its Applications, vol. 20, Cambridge University Press, 1997.
- [8] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [9] I. J. Good, “The interaction algorithm and practical Fourier analysis,” *Journal of the Royal Statistical Society, Series B*, vol. 20, pp. 361–372, 1958.
- [10] L. H. Thomas, “Using a computer to solve problems in physics,” in *Applications of Digital Computers*, Ginn, Boston, 1963.
- [11] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Cryptographic sponge functions,” Submission to NIST, 2011. Available: <https://keccak.team/files/SpongeFunctions.pdf>
- [12] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, Internet Engineering Task Force (IETF), 2010.
- [13] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland, Amsterdam, 1977.
- [14] P. J. Davis, *Circulant Matrices*, John Wiley & Sons, New York, 1979.
- [15] A. Canteaut, S. Carpov, C. Fontaine, T. Lepoint, M. Naya-Plasencia, P. Paillier, and R. Sirdey, “Stream ciphers: A practical solution for efficient homomorphic-ciphertext compression,” in *Fast Software Encryption (FSE 2016)*, Lecture Notes in Computer Science, vol. 9783, pp. 313–333, Springer, 2016.
- [16] A. Banerjee, H. Brenner, G. Leurent, C. Peikert, and A. Rosen, “SPRING: Fast pseudorandom functions from rounded ring products,” in *Fast Software Encryption (FSE 2014)*, Lecture Notes in Computer Science, vol. 8540, pp. 38–57, Springer, 2014.
- [17] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM,” in *IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 353–367, 2018.
- [18] E. Biham and A. Shamir, “Differential cryptanalysis of DES-like cryptosystems,” *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1991.
- [19] M. Matsui, “Linear cryptanalysis method for DES cipher,” in *EUROCRYPT 1993*, Lecture Notes in Computer Science, vol. 765, pp. 386–397, Springer, 1993.
- [20] A. Rukhin et al., “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” NIST Special Publication 800-22 Rev. 1a, 2010.
- [21] P. L’Ecuyer and R. Simard, “TestU01: A C library for empirical testing of random number generators,” *ACM Transactions on Mathematical Software*, vol. 33, no. 4, pp. 1–40, 2007.
- [22] P. Flajolet and A. Odlyzko, “Random mapping statistics,” in *EUROCRYPT 1989*, Lecture Notes in Computer Science, vol. 434, pp. 329–354, Springer, 1990.
- [23] D. J. Bernstein, “ChaCha, a variant of Salsa20,” in *Workshop Record of SASC 2008*, 2008.
- [24] E. B. Wilson and M. M. Hilferty, “The distribution of chi-square,” *Proceedings of the National Academy of Sciences*, vol. 17, no. 12, pp. 684–688, Dec. 1931.
- [25] J. von Neumann, “Various techniques used in connection with random digits,” in *Monte Carlo Method*, National Bureau of Standards Applied Mathematics Series, vol. 12, pp. 36–38, U.S. Government Printing Office, Washington, D.C., 1951.
- [26] Intel Corporation, “Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 2B: Instruction Set Reference, M-U,” Chapter 17.17, 2015.